

SAAS-001 Blueprint

How I Automated My Entire YouTube Script Pipeline with Claude + Node.js

Generated for SaaS Autopilot on 2026-06-07.

Purpose

I built a system that writes, voices, edits, and schedules YouTube videos while I sleep. Here's every piece of it.

Workflow Stack

- claude

Build Outline

Scene	Purpose	Viewer takeaway
1	The Hook (0:00-0:30)	I built an autonomous system that writes, voices, edits, and schedules YouTube videos while I sleep. There are no virtual assistants, no human editors, and no manual uploading. In this video, I'm going to pull back the curtain and show you the exact system, code, and prompts running the entire channel portfolio behind the scenes. Let's get straight into it.
2	The Problem (0:30-1:30)	Content creation is an operational nightmare. For a typical faceless channel, the workflow is highly fragmented. First, you spend hours researching topics and writing scripts. Then, you record or generate voiceovers. Next, you spend days inside an editor manually placing b-roll, syncing audio, and cutting transitions. Finally, you have to format the metadata, generate thumbnails, and schedule the upload in YouTube Studio. This manual process takes between 8 to 15 hours per video. It is repetitive, exhausting, and highly prone to human error, making it almost impossible to scale a portfolio of channels consistently.
3	The Stack (1:30-2:30)	To automate this, we are using a highly optimized, low-cost stack. The brains of the operation is Claude Sonnet via the Anthropic API, orchestrated by a local Node.js runtime. We use ElevenLabs for high-fidelity voice synthesis, and ffmpeg for programmatic video stitching and editing. The entire system is managed via local JSON configurations and automated state files. There are no expensive no-code subscription tools-only clean, local code. A full video costs about \$0.40 to write, voice, and compile.
4	The Build (2:30-6:00)	Let's walk through the architecture. The system is split into three modular pipelines. Pipeline one is script generation. The script writer agent reads our topic backlog from a central JSON config and produces a structured scene-by-scene script scaffold, strictly enforcing pacing, depth, and word count rules. Pipeline two is voiceover synthesis. The TTS agent processes the script JSON, segments the text by scene, and calls the ElevenLabs API using a consistent voice ID to generate perfect audio files. Pipeline three is the automated editor. Our editing agent reads the visual plans and timestamps, programmatically overlays the corresponding screen captures and assets, runs audio ducking for background music, and runs ffmpeg to stitch the final MP4. The upload safety agent then validates the video against key publishing gates before utilizing the YouTube Data API to schedule it.
5	The Results (6:00-7:00)	The results are incredible. What used to be a 10-hour manual process is now a 5-minute automated pipeline. I run a single command in my terminal, and the agents handle the rest. The system generates a complete, monetization-ready, 9-minute video scheduled directly to YouTube with custom metadata and transcripts. Operating cost is virtually zero, and it allows us to run multiple niche channels concurrently with flawless scheduling integrity.

Scene	Purpose	Viewer takeaway
6	The CTA (7:00-end)	This entire codebase is fully open and documented. You can find the GitHub link to the repository in the description below, along with the system prompts we use for scriptwriting. If you want to build this yourself, clone the repo and follow the setup guide. Drop a comment below if you have any questions or want to see a specific workflow automated next, and for real, no-hype automation tutorials. This channel needs your support. Please subscribe, leave a word of appreciation for the author, and may God bless you.

Implementation Notes

- Start with a test account and fake sample data.
- Replace all placeholders before connecting live accounts.
- Keep human review before sending outbound messages or changing production systems.
- Record a successful dry run before publishing the workflow to viewers.

Prompt-Improved Build Prompt

Prompt-Improved Automation Build Template

Optimized locally with the SaaS Autopilot prompt improver rules: clear role, context, variables, workflow, constraints, QA, and output format.

ROLE & PERSONA

You are a senior B2B automation architect, prompt engineer, and implementation QA lead.

CONTEXT & OBJECTIVE

Build a practical automation blueprint for: How I Automated My Entire YouTube Script Pipeline with Claude + Node.js

The workflow should help {{target_customer}} reduce {{manual_task}} and produce {{business_outcome}} using this likely stack: claude.

REQUIRED INPUT VARIABLES

- {{target_customer}}
- {{workflow_goal}}
- {{manual_task}}
- {{business_outcome}}
- {{tool_stack}}
- {{data_source}}
- {{approval_owner}}
- {{risk_level}}

STEP-BY-STEP WORKFLOW

1. Summarize the business problem in plain language.
2. Map each input source, transformation step, human review point, and final output.
3. Recommend the exact automation modules or API steps needed for {{tool_stack}}.
4. Create sample test data that avoids real customer information.
5. Define error handling for missing data, failed API calls, duplicate records, and bad outputs.
6. Add a human approval checkpoint before any outbound message, billing change, or production update.
7. Provide a go-live plan with monitoring for the first three live runs.

CONSTRAINTS & SAFETY RULES

- Do not invent credentials, private customer data, or unverifiable claims.
- Keep outbound communication respectful, short, and compliant with platform rules.
- Prefer test accounts and fake data until QA passes.
- Include rollback steps for every production action.

OUTPUT FORMAT

Return the answer with these sections:

1. Workflow Summary
2. Tools and Accounts Needed
3. Step-by-Step Build Plan
4. Prompt or Message Templates
5. Test Data
6. QA Checklist
7. Go-Live Checklist
8. Risks and Mitigations